Universidad Nacional de Colombia
Facultad de Ingeniería



Estructuras lineales

Presentado por:
Julian Felipe Acosta Pineda
Lizeth Mariana Rubiano Torres
Andres Julian Sanchez Escobar

Profesor:
Jonatan Gomez Perdomo

Asignatura:
Estructuras de Datos - G06

26 de Octubre de 2026
Bogotá, D.C.

Tabla de contenidos

1. Introducción..... 3
1.1. Descripción del proyecto.....*3*
1.2. Objetivo del proyecto..... *3*
1.3. Uso de inteligencia artificial.....*3*
2. Diseño general del sistema..... 3
2.1. Flujo del restaurante.....*3*
2.2. Resumen de estructuras usadas..... *4*
3. Estructuras de datos.....5
3.1. Arreglos dinámicos (mesas y menú).....*5*
3.2. Pila (comanda de cada mesa).....*6*
3.3. Cola normal (fila de espera común)..... *7*
3.4. Cola con prioridad (fila VIP)..... *8*
3.5. Cola circular (meseros y domiciliarios)..... *9*
4. Pruebas y resultados..... 10
5. Conclusiones..... 11

1. Introducción

1.1. Descripción del proyecto

El proyecto realizado consiste de un programa de gestión integral de un restaurante, en lenguaje de programación C++, en el cual se emplean distintas estructuras de datos con el fin de organizar las funciones del mismo, como lo son la administración de clientes, mesas y pedidos; y logrando su funcionamiento a través de una interfaz de usuario cómoda y eficaz.

La principal utilidad del programa es que los administradores de restaurantes y locales de comida puedan gestionar su negocio de forma fácil y rápida, sin necesidad de conocimientos previos ni de personas externas.

A lo largo del programa se utilizaron arreglos dinámicos (menú y mesas), pilas/stacks (comandas de meseros), colas simples (clientes estándar), colas con prioridad (clientes VIP), y colas circulares (gestión de personal); agrupando todos estos elementos de forma óptima para una correcta ejecución del proyecto.

1.2. Objetivo del proyecto

Diseñar e implementar un sistema de gestión especializado en restaurantes, permitiendo a los administradores manejar la asignación de mesas, la toma y corrección de pedidos, control de clientes, rotación de personal, etc., integrando 5 estructuras de datos lineales y desarrollando una interfaz de usuario cómoda e intuitiva para cualquier persona sin conocimientos de programación o informática.

1.3. Uso de inteligencia artificial

Durante el desarrollo se utilizaron herramientas de inteligencia artificial, como chatbots o LLMs, principalmente para diagnóstico y corrección de errores y comentarios, además de explicaciones teóricas para la resolución de los problemas. A lo largo del proyecto, la inteligencia artificial colaboró alrededor de un 40%.

2. Diseño general del sistema

2.1. Flujo del restaurante

El programa sigue el recorrido de un cliente desde que llega a la puerta hasta que deja la mesa, y al mismo tiempo atiende los pedidos a domicilio.

Todo empieza cuando el host registra al cliente. Si tiene tarjeta de fidelización pasa a la fila VIP, si no, espera en la fila común. Cuando se llama al siguiente, el sistema aplica una regla sencilla: por cada dos clientes VIP se atiende uno de la fila común, de modo que nadie quede esperando indefinidamente. Una vez llamado, el cliente recibe una mesa libre, el mesero que sigue en turno toma la comanda plato por plato y cuando el cliente se retira, la mesa se libera. El Diagrama 1 resume el proceso que se sigue en el salón.

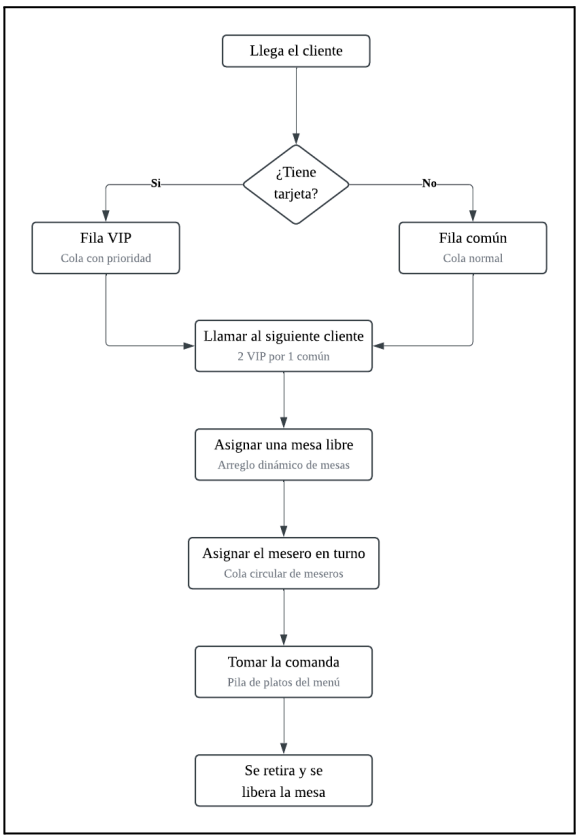


Figura 1. Diagrama de flujo de la gestión del salón

Por otra parte, los domicilios siguen un camino paralelo: se arma el pedido, se asigna al siguiente domiciliario en turno, se entrega y se libera el repartidor. El Diagrama 2 resume el proceso que se sigue en cuanto a los domicilios.

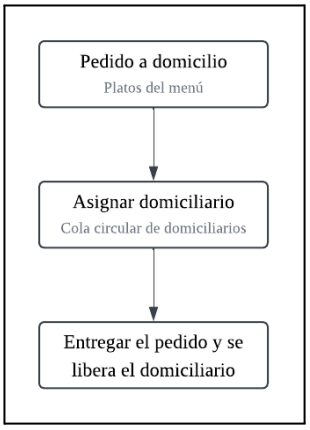


Figura 2. Diagrama de flujo de la gestión de los domicilios

2.2. *Resumen de estructuras usadas*

Tabla 1. Resumen de estructuras usadas en el programa Orderstack

Estructura	¿Dónde se usa?	¿Cómo se usa?
Arreglo dinámico	Mesas y menú	Guarda las mesas del salón y los platos. Se consulta por posición y crece si se agregan más.

Pila (stack)	Comanda de la mesa	Los platos se apilan en el orden pedido. El último en entrar es el primero en retirarse si el cliente decide cambiar su pedido.
Cola normal	Fila de espera común	Clientes sin tarjeta: Atendidos por orden de llegada.
Cola con prioridad	Fila de espera VIP	Clientes con tarjeta: Ordenados por nivel de tarjeta y, si tienen igual nivel, por orden de llegada.
Cola circular	Meseros y domiciliarios	Turnos rotativos: Cuando atiende el último, el turno vuelve al primero.

3. Estructuras de datos

A continuación, se presenta cada estructura utilizada en el programa. Para cada una de ellas resolveremos las siguientes preguntas: qué problema se resuelve dentro del restaurante, la razón por la que la elegimos, un fragmento breve de su implementación y el funcionamiento y complejidad de sus operaciones principales.

3.1. Arreglos dinámicos (mesas y menú)

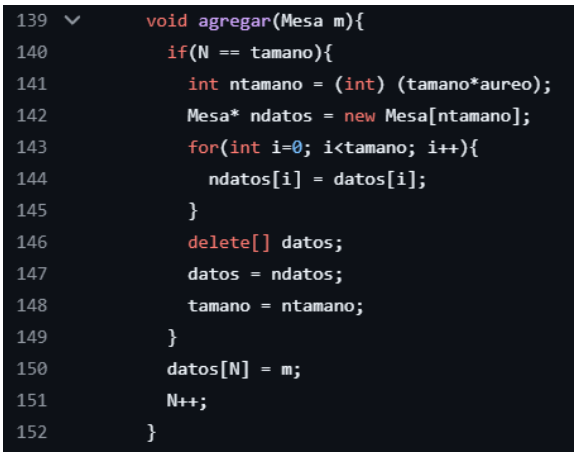
- *¿Qué problema resuelve?* El restaurante maneja dos listas que cambian con el tiempo, estas son las mesas del salón, que pueden ampliarse en días de mucha afluencia, y los platos del menú, que se agregan o se retiran. Un arreglo de tamaño fijo obligaría a decidir desde el principio cuántos elementos habrá, y esa cifra no se conoce con certeza.

- *¿Por qué esta estructura?* Tanto las mesas como los platos se consultan por posición (la mesa 7, el plato 3), y el arreglo ofrece ese acceso directo en tiempo constante. Como la cantidad final es incierta, elegimos la variante dinámica, que amplía su capacidad multiplicándola por la razón áurea ($\varphi \approx 1.618$) cuando se llena, en lugar de duplicarla. Una lista enlazada habría obligado a recorrer nodo por nodo para llegar a la mesa 7, sin ofrecer ninguna ventaja en este caso.

- *¿Cómo se implementó?*

```
68      void agregar(Plato p){
69        if(N == tamano){
70          int ntamano = (int) (tamano*aureo);
71          Plato* ndatos = new Plato[ntamano];
72          for(int i=0; i<tamano; i++){
73            ndatos[i] = datos[i];
74          }
75          delete[] datos;
76          datos = ndatos;
77          tamano = ntamano;
78        }
79        datos[N] = p;
80        N++;
81      }
```

Figura 3. Código del método "agregar(Plato p)"



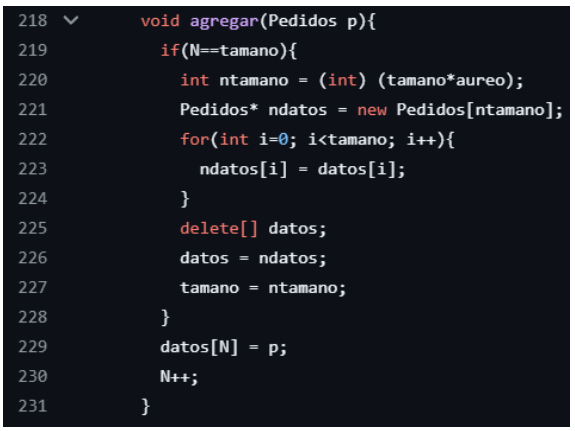
```
139     void agregar(Mesa m){
140       if(N == tamano){
141         int ntamano = (int) (tamano*aureo);
142         Mesa* ndatos = new Mesa[ntamano];
143         for(int i=0; i<tamano; i++){
144           ndatos[i] = datos[i];
145         }
146         delete[] datos;
147         datos = ndatos;
148         tamano = ntamano;
149       }
150       datos[N] = m;
151       N++;
152     }
```

Figura 4. Código del método "agregar(Mesa m)"

- *¿Cómo funcionan sus operaciones?*
    - Acceder a un elemento por posición: tiempo constante, O(1).
    - Agregar al final: O(1) en la mayoría de los casos. Solo cuando el arreglo está lleno hay que copiar todo a uno más grande, lo que cuesta O(n) pero ocurre pocas veces.
    - Insertar o eliminar en medio: O(n), porque hay que desplazar los elementos siguientes.

3.2. Pila (comanda de cada mesa)

- *¿Qué problema resuelve?* Cuando el mesero toma el pedido de una mesa va anotando un plato tras otro, y puede suceder que el cliente se arrepienta de lo último que pidió. Para ello se necesitó una estructura que guarde los platos en el orden en que se piden y que permita deshacer el más reciente sin complicaciones.
- *¿Por qué esta estructura?* La pila funciona bajo la regla LIFO (Last In First Out, el último en entrar es el primero en salir), que coincide con la forma natural de corregir una comanda: se cancela lo último que se agregó. Además, cada mesa tiene su propia pila.
- *¿Cómo se implementó?*



```
218     void agregar(Pedidos p){
219       if(N==tamano){
220         int ntamano = (int) (tamano*aureo);
221         Pedidos* ndatos = new Pedidos[ntamano];
222         for(int i=0; i<tamano; i++){
223           ndatos[i] = datos[i];
224         }
225         delete[] datos;
226         datos = ndatos;
227         tamano = ntamano;
228       }
229       datos[N] = p;
230       N++;
231     }
```

Figura 5. Código del método "agregar(Pedidos p)"

```
233     void borrar(){
234       if(N>0){
235         if(N*aureo*aureo<tamano){
236           int ntamano = (int) (N*aureo);
237           if(ntamano<10) ntamano = 10;
238           Pedidos* ndatos = new Pedidos[ntamano];
239           for(int i=0; i<N; i++){
240             ndatos[i] = datos[i];
241           }
242           delete[] datos;
243           datos = ndatos;
244           tamano = ntamano;
245         }
246         N--;
247       }
248     }
```

Figura 6. Código del método "borrar"

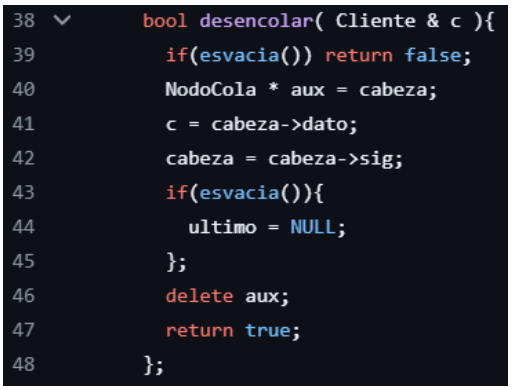
- *¿Cómo funcionan sus operaciones?*
    - Apilar (agregar un plato al tope): O(1).
    - Desapilar (quitar el último plato pedido): O(1).
    - Consultar el tope sin retirarlo: O(1).
    - Verificar si está vacía: O(1). Todas las operaciones trabajan solo sobre el tope, por eso no dependen de cuántos platos haya.

3.3. Cola normal (fila de espera común)

- *¿Qué problema resuelve?* Los clientes sin tarjeta VIP llegan al restaurante y quedan a la espera de una mesa disponible. Se requiere un mecanismo de atención justo que garantice el respeto estricto del orden de llegada, asegurando que ningún cliente sea atendido antes que otro que haya ingresado previamente a la fila.
- *¿Por qué esta estructura?* La cola sigue la regla FIFO (First In First Out, el primero en entrar es el primero en salir), que es exactamente el criterio de una fila de espera. La implementamos con nodos enlazados y guardamos un puntero al último nodo, así agregar al final no exige recorrer toda la fila.
- *¿Cómo se implementó?*

```
28     void encolar( Cliente c ){
29       NodoCola * aux = new NodoCola(c);
30       if(esvacia()){
31         cabeza = aux;
32       }else{
33         ultimo->sig = aux;
34       };
35       ultimo = aux;
36     };
```

Figura 7. Código del método "encolar(Cliente c)" en una cola común



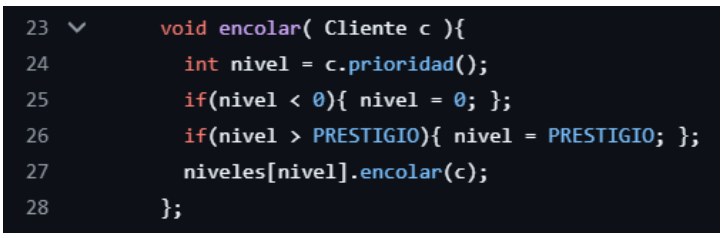
```
38      bool desencolar( Cliente & c ){
39        if(esvacia()) return false;
40        NodoCola * aux = cabeza;
41        c = cabeza->dato;
42        cabeza = cabeza->sig;
43        if(esvacia()){
44          ultimo = NULL;
45        };
46        delete aux;
47        return true;
48      };
```

Figura 8. Código del método "desencolar(Cliente & c)" en una cola común

- *¿Cómo funcionan sus operaciones?*
    - Encolar (un cliente llega y se pone al final): O(1).
    - Desencolar (se llama al primero): O(1).
    - Consultar al primero sin sacarlo: O(1).
    - Conocer la longitud de la fila: O(n), porque se recorre contando nodos.

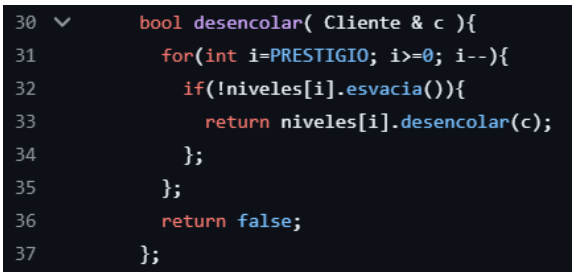
3.4. Cola con prioridad (fila VIP)

- *¿Qué problema resuelve?* El host entrega tarjetas de fidelización a ciertos clientes: frecuentes, amigos del gerente y personas con prestigio. Ellos deben pasar antes que el resto, y entre ellos también hay niveles, pero dos clientes del mismo nivel no deben saltarse el orden de llegada.
- *¿Por qué esta estructura?* Para implementar la cola con prioridad usamos un arreglo de colas: cada posición corresponde a un nivel de tarjeta (1, 2 o 3) y guarda la cola de los clientes de ese nivel. Cada cliente se pone al final de la cola de su nivel, así que a igual nivel se respeta el orden de llegada. Para atender, se revisan los niveles de mayor a menor y sale el primero del primer nivel que tenga gente. Finalmente, para que la fila común no se quede esperando para siempre, el programa principal alterna dos clientes VIP y luego uno común.
- *¿Cómo se implementó?*



```
23      void encolar( Cliente c ){
24        int nivel = c.prioridad();
25        if(nivel < 0){ nivel = 0; };
26        if(nivel > PRESTIGIO){ nivel = PRESTIGIO; };
27        niveles[nivel].encolar(c);
28      };
```

Figura 9. Código del método "encolar(Cliente c)" en una cola VIP



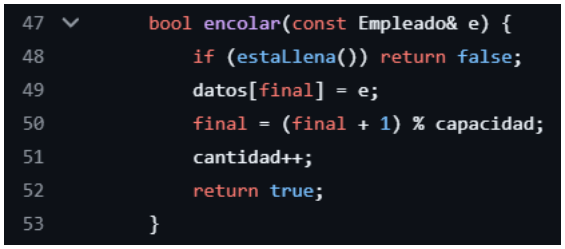
```
30      bool desencolar( Cliente & c ){
31        for(int i=PRESTIGIO; i>=0; i--){
32          if(!niveles[i].esvacia()){
33            return niveles[i].desencolar(c);
34          };
35        };
36        return false;
37      };
```

Figura 10. Código del método "desencolar(Cliente & c)" en una cola VIP

- *¿Cómo funcionan sus operaciones?*
    - Encolar: O(1).
    - Desencolar: O(k), con k el número de niveles. Como son 3 y fijos, es constante en la práctica. Asimismo, consultar al primero y verificar si está vacía, también es de O(k).
    - Mostrar la fila: O(n).

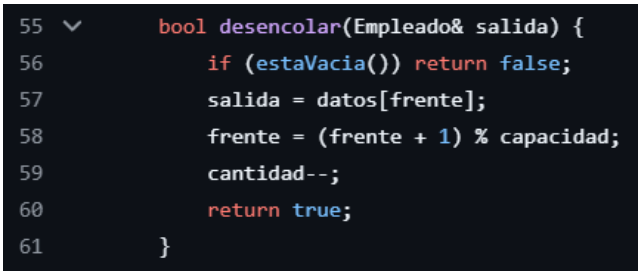
3.5. Cola circular (meseros y domiciliarios)

- *¿Qué problema resuelve?* Tanto los clientes como los pedidos a domicilio deben repartirse entre varias personas. Si la asignación siempre empezara por el primero de la lista, ese mesero o ese domiciliario cargaría con casi todo el trabajo y los demás quedarían sin atender.
- *¿Por qué esta estructura?* Una cola circular se apoya en un arreglo de tamaño fijo, algo razonable aquí porque el número de meseros y de domiciliarios es limitado y conocido. Sus índices avanzan con el operador módulo, de modo que al llegar al final del arreglo vuelven al principio sin mover ningún elemento. Esa propiedad produce un turno rotativo: quien atiende pasa al final de la fila y el trabajo se reparte de manera equitativa.
- *¿Cómo se implementó?*



```
47      bool encolar(const Empleado& e) {
48          if (estaLlena()) return false;
49          datos[final] = e;
50          final = (final + 1) % capacidad;
51          cantidad++;
52          return true;
53      }
```

Figura 11. Código del método "encolar(Empleado & e)" en una cola circular



```
55      bool desencolar(Empleado& salida) {
56          if (estaVacia()) return false;
57          salida = datos[frente];
58          frente = (frente + 1) % capacidad;
59          cantidad--;
60          return true;
61      }
```

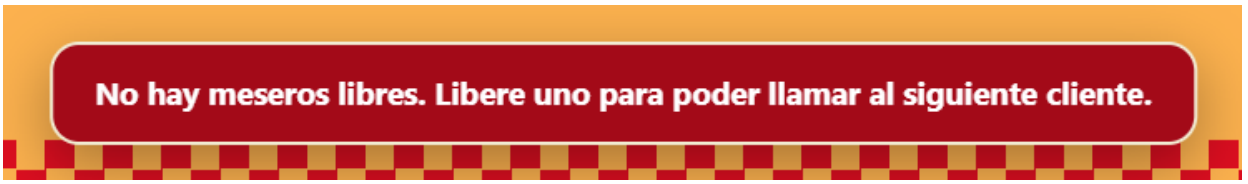
Figura 12. Código del método "desencolar(Empleado & salida)" en una cola circular

- *¿Cómo funcionan sus operaciones?*
    - Encolar, desencolar y rotar el turno: O(1), pues solo se actualizan índices.
    - Verificar si está llena o vacía: O(1), con ayuda del contador de elementos.

4. Pruebas y resultados

Las pruebas se hicieron ejecutando el programa en Visual Studio, que recorre todo el programa en orden. Cada ejecución empieza con los datos iniciales de la sección 2.1.

Tabla 2. Pruebas

N.º	Qué se prueba	Qué se hace	Resultado
1	Prioridad en la fila VIP	Salón, Registrar: Luis (tarjeta 1) y después Lionel Messi (tarjeta 3).	Messi quedó primero en la fila VIP, aunque llegó después.
			
Figura 13. Prueba 1			
2	Sin meseros libres	Llamar a un cliente con los 3 meseros ocupados	«No hay meseros libres» y el cliente sigue esperando
			
Figura 14 . Prueba 2			
3	Grupo que no cabe en ninguna mesa	Salón, Registrar cliente: 10 personas	«Ninguna mesa tiene capacidad para 10 personas (máximo: 6)»
			
Figura 15 . Prueba 3			

4	Comanda y deshacer (pila)	Mesa 1: hamburguesa, limonada y helado. Deshacer, helado	Se deshace el helado, que era el último



Figura 16 . Prueba 4

5	Arreglo dinámico del menú	Administración, Menú, agregar Pizza $15000	La pizza aparece al final del menú

POS.	PLATO
[0]	Hamburguesa Artesanal
[1]	Alitas BBQ (x10)
[2]	Limonada de Coco
[3]	Helado de Vainilla

POS.	PLATO
[0]	Hamburguesa Artesanal
[1]	Alitas BBQ (x10)
[2]	Limonada de Coco
[3]	Helado de Vainilla
[4]	Pizza

Figura 17 . Prueba 5

6	Quitar platos con el menú vacío	Quitar el último plato varias veces hasta vaciarlo y una vez más	«El menú está vacío», sin errores

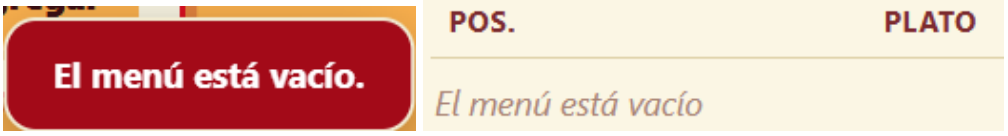


Figura 18 . Prueba 6

5. Conclusiones

Orderstack cumplió el objetivo planteado, obtuvimos un programa en C++ que gestiona el funcionamiento de un restaurante a partir de cinco estructuras lineales, cada una asignada a un problema concreto del negocio. Los arreglos dinámicos guardan las mesas y el menú, la pila maneja la comanda de cada mesa, la cola normal y la cola con prioridad organizan las filas de espera común y VIP, y la cola circular reparte el trabajo entre meseros y domiciliarios.

El principal aprendizaje fue que la estructura se elige a partir del problema y no al revés; por ejemplo, el acceso directo por posición justificó el arreglo, el orden inverso de corrección de una comanda justificó la pila (LIFO), y la necesidad de un turno rotativo justificó la cola circular. También vimos que toda decisión tiene un costo: dividir la fila VIP

en una cola por nivel hace que insertar sea O(1), pero obliga a revisar los niveles al atender (solo tres, así que el costo sigue siendo constante), y que la atención al cliente tenga una complejidad de O(1). Por otra parte, otro aprendizaje fundamental que obtuvimos es que el VIP por sí solo podía dejar sin atender a la fila común, pero lo resolvimos con la regla de dos clientes VIP por cada cliente común, esto mostró que el diseño depende tanto de cada estructura como de la forma en que se combinan.

Finalmente, como mejoras a futuro, podríamos guardar las mesas, el menú y los pedidos en archivos para no perder la información al cerrar el programa, y desarrollar una interfaz gráfica más interactiva para mejorar la experiencia del usuario. En conjunto, el proyecto nos permitió tener una aplicación completa de las estructuras lineales vistas en clase.